CS 225 - Data Structures

Quiz 1

Spring 2026

Thursday, Jan 22nd, 2026

Name	Hanbin Zheng
ZJU ID	
Lab TA	

- **Be sure that your exam booklet has EXACTLY () pages.**
- **Write your name, ZJU ID, and your lab TA's name on the title page above.**
- **Do not tear the exam booklet apart.**
- **This is a closed book and closed notes exam. No electronic aids are allowed, either.**
- **Absolutely no interaction between students is allowed.**
- **Clearly indicate any assumptions that you make.**
- **The questions are not weighted equally. Budget your time accordingly.**
- **Unless the problem specifically says otherwise, assume the code compiles.**
- **Don't panic and good luck!**

Problem 1	10 points	9.5
Problem 2	10 points	10
Problem 3	10 points	10
Problem 4	10 points	8.5
Total	40 points	38

Problem 1 (10 points): Memory

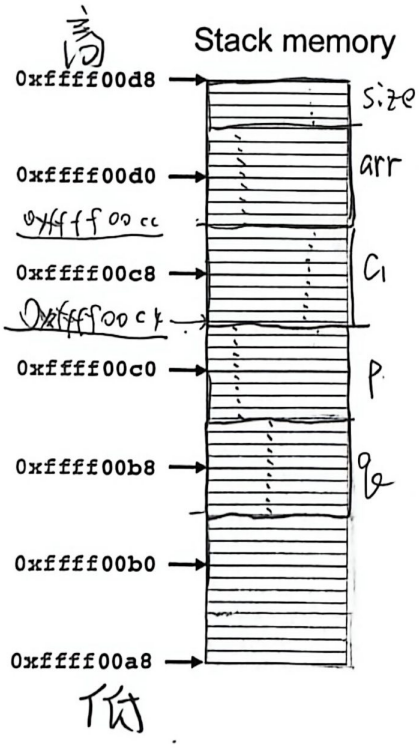
Suppose we've declared class Circle as:

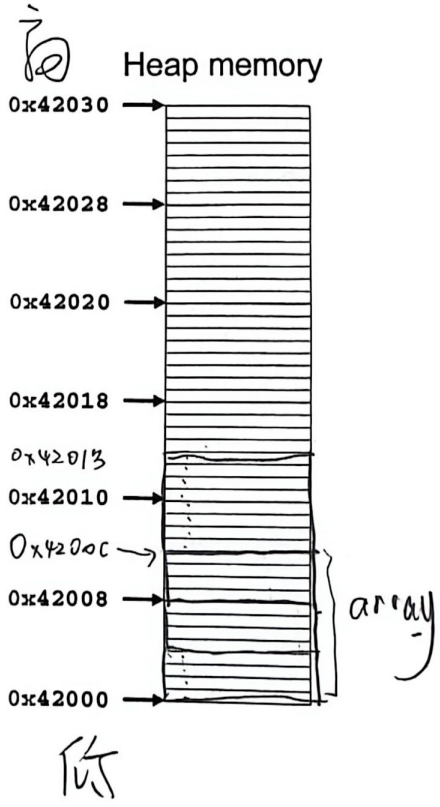
```
class Circle {
  public:
    Circle(double r = 1.0) : r_(r) { }
    double getRadius() { return r_; }
    void setRadius(double r) { r_ = r; }
  private:
    double r_;
};
```

Consider the following statements:

```
int size = 3;
int *arr = new int[size];
Circle c1(2.0);
Circle *p = &c1;
Circle *q = new Circle(4.0);
q->setRadius(2.0);
```

You can assume that size of `int` is 4 bytes, size of `double` is 8 bytes, and size of pointer type is 8 bytes. Stack memory starts at address `0xfff00d8` and heap memory starts at address `0x42000`.





I assume that, on heap, the memory is still tightly allocated.

a) List all variables and objects that are stored on the stack during the execution of this code. For each, specify its name, its type, and the value it holds (for example, a concrete number or a pointer to what kind of object/array).

on stack.

name	type	value
size	int	3
arr	array/int*	the first address of corresponding array on heap (0x42000)
C1	circle object	C{r_ = 2.0}
p	circle*	the address of C1 (0xffff00C4)
q	circle*	the address of corresponding circle object on the heap (0x4200C)

on heap.

name	type	value
unknown content of arr	int	unknown
unknown content of *q	circle object	{r_=4.0}

b) List all memory blocks that are allocated on the heap in this code. For each block, specify which statement allocates it, what data is stored in it

heap: ① 0x42000 ~ 0x4200B (12 byte)
the int array. code: int *arr = new int[size];
data: unknown

② 0x4200C ~ 0x42013 (8 byte)
data: {r_=4.0} code: Circle *q = newCircle(4.0);

c) What error will occur if we directly "return 0" after this piece of code: memery leak.
we new an int array and a circle object on the heap, but we never delete[] it.
memory

d) What code do you need to add before "return 0" to avoid this problem? (two lines will be sufficient)

delete[] arr; arr = nullptr;
delete[] q; q = nullptr;

delete q;

delete[] is only used for array.

Problem 2 (10 points): Pointers

a) Consider the following statement. What is the final output?

```
int a = 2;
int b = 5;
int *p = &a;
int *q = &b;
*q = *p + 3;
p = q;
*p += 2;
q = &a;
*q = *p - 1;
cout << a << " " << b << endl;
```

(a) 6 7
(b) 5 6
(c) 5 7
(d) 6 5

b) Consider the following statement. What is the final output?

```
int x = 10;
int arr[4] = {1,2,3,4};
int *p = arr;
int *q = &arr[2];
p = p + 1;
*q = *(p + 2) + x;
*(q - 1) = *p + 5;
cout << arr[0] << " " << arr[1] << " " << arr[2] << " " << arr[3]
<< endl;
```

arr[0]=_____

arr[1]=_____

arr[2]=_____

arr[3]=_____

c) What common bug is shown in the following code snippet?

```
int * foo;
*foo = 5;
```

(a) syntax error
(b) type mismatch
(c) uninitialized pointer
(d) dereferenced NULL pointer
(e) None of the above

this bug can also be "dereferenced NULL pointer", since the unintialized pointer "foo" will be initialized as "nullptr" automatically.

add of a.

reference.

d) Consider the following statement. What is the final output?

```
void foo(int *p, int &x) {
    *p += 3;
    x = *p - 1;
     p = &x;
    *p += 2;
}

int main() {
    int a = 4;
    int b = 7;
    foo(&a, b);
    cout << a << " " << b << endl;
}
```

a=7 b=8

a=7=4+3

b=a-1=7-1=6

p points to b.

b=b+2=6+2=8

a=7

b=6+2=8

a=4 b=7

a+=3 a∈7

b=a-1 b∈6.

b+=2 b∈8.

Problem 3 (10 points): Classes and objects

Consider the following class:

```
class BankAccount {
  public:
    // Initializes member variables to the given parameters.
    BankAccount(double initialBalance, std::string ownerName);

    // Returns the balance of the account
    double getBalance();

    // Deposit operation, increases the account balance
    void deposit(double amount);

    // Withdrawal operation, decreases the account balance
    void withdraw(double amount);

  private:
    double balance;          // Account balance
    std::string owner;       // Account holder's name
};
```

Consider the following code:

```
BankAccount * accountPtr;
```

Given the variable `accountPtr` above, write additional code that will accomplish the following:

- Create a local (on the runtime stack) BankAccount object with an initial balance of 500.0 and the account holder's name "Robert".
- Assign the pointer accountPtr to point to this BankAccount object.
- Using the pointer, retrieve the balance of the account and print it out using std::cout. ✓
- Using the pointer, perform a deposit operation that increases the balance by 200.0 and then perform a withdrawal operation that decreases the balance by 100.0.

The implementation of class BankAccount is attached on the next page, and the code is shown below.

```
BankAccount * accountPtr;
BankAccount local_obj = BankAccount(500.0, "Robert");
accountPtr = &local_obj;
double balance = accountPtr->getBalance();
std::cout << balance << std::endl;
accountPtr -> deposit(200.0);
accountPtr -> withdraw(100.0);
```

```
BankAccount::BankAccount(double initialBalance, std::string ownerName):
    balance(initialBalance), owner(ownerName) { ; }

double BankAccount::getBalance() {
        return balance;
    }

void BankAccount::deposit(double amount) {
        balance += amount;
    }

void BankAccount::withdraw(double amount) {
        balance -= amount;
    }
```

Problem 4 (10 points): lab_intro

(1) Shown below are snippets of code from lab_intro:

```
class HSLAPixel {
  ...
  HSLAPixel(double hue, double sat, double lum, double
  alpha = 1.0);
  private:
    double h, s, l, a;
};
```

default value : only once, in declaration not definition

FInish the code below such that the the constructor uses initialization list to initialize all members.

-1

```
HSLAPixel::HSLAPixel(double hue, double sat, double lum, double alpha=1.0) :
  h(hue), s(sat), l(lum), a(alpha) {
        ; // empty implementation
}
```

(2)

7.5

Alma is too shy—he doesn't want to appear clearly in front of everyone. Please help him implement a box blur function(You may assume that no error checking is required)(Tip: You can access references (`PNG &image`) the same way you normally do—no need to worry about this detail.):

```
// Compute the box-blurred pixel value at (x, y) from the
// source image,using a (2*radius+1)x(2*radius+1) neighborhood
// (clamped * at borders).This returns a new HSLAPixel value
// (does not modify the source image).

HSLAPixel boxBlurPixel(PNG &image, unsigned x, unsigned y, int
radius) {
  int w = (int)image.width();
  int h = (int)image.height();

  int x0 = (int)x - radius;
  int x1 = (int)x + radius;
  int y0 = (int)y - radius;
  int y1 = (int)y + radius;

  if (x0 < 0) x0 = 0;
  if (y0 < 0) y0 = 0;
```

```
  if (x1 >= w) x1 = w - 1;
  if (y1 >= h) y1 = h - 1;

  double sumS = 0.0, sumL = 0.0, sumA = 0.0;
  double sumCos = 0.0, sumSin = 0.0;  // circular mean for hue
  int count = 0;

// Iterate over all pixels in the clamped neighborhood window
// and accumulate. INCLUDE the borders(eg. When radius = 1,
// you should iterate over 9 pixels)
for(int i=x0; i<=x1; i++) {
  for(int j=y0; j<=y1; j++) {
    // Read a pixel from the original image (source)
    HSLAPixel *p = image.getPixel(i, j);

      double rad = p->h * M_PI / 180.0;
      sumCos += cos(rad);
      sumSin += sin(rad);

      // Accumulate linear channels directly.
      sumS += p->s;
      sumL += p->l;
      sumA += p->a;
      count++;
    }
  }

  HSLAPixel out;  // default constructed
  double avgH = atan2(sumSin, sumCos) * 180.0 / M_PI;
  if (avgH < 0) avgH += 360.0;

  out.h = avgH;
  // Write averaged channels into output pixel.
  out.s = sumS / (double)count;
  out.l = sumL / (double)count;
  out.a = sumA / (double)count;

  return out;
}
```

为什么不上机考，狗旦们

-0.5

// sorry that I forgot the interface of PNG class,